

RETO 01

Integridad y calidad de
datos en sistemas
distribuidos

Antonio Ferrer Martínez

01_Tarea_Calidad_IoT

March 10, 2026

1 Tarea Práctica: Calidad de Datos en IoT (Smart Building)

1.1 Contexto

Estás trabajando como Data Engineer para una empresa de edificios inteligentes. Tenéis una red de sensores de temperatura y humedad desplegada en las oficinas. Sin embargo, la red es inestable: a veces los sensores pierden conexión, envían datos duplicados o se descalibran y mandan valores imposibles.

1.2 Objetivo

Tu misión es limpiar el dataset raw (sucio) para que pueda ser usado por el equipo de Data Science para predecir el consumo energético.

Debes resolver los siguientes problemas: 1. **Compleitud:** Gestionar los datos faltantes. 2. **Consistencia:** Eliminar duplicados. 3. **Precisión:** Filtrar valores erróneos (Outliers).

1.2.1 !IMPORATANTE, SI TOMAS UNA DECISIÓN , QUIERO QUE ME LA JUSTIFIQUES. NOTA: SI HAY IA, TENDRAS UN PROBLEMA, You know what I mean.!

```
[1]: import pandas as pd
import numpy as np

# --- CÓDIGO DE GENERACIÓN DE DATOS (NO MODIFICAR) ---

data = {
    'sensor_id': ['S-01', 'S-02', 'S-01', 'S-03', 'S-04', 'S-01', 'S-05', 'S-02'],
    'timestamp': [
        '2024-02-10 09:00', '2024-02-10 09:00',
        '2024-02-10 09:15', # S-01 lectura correcta
        '2024-02-10 09:15',
        '2024-02-10 09:15',
        '2024-02-10 09:00', # S-01 DUPLICADO (mismo timestamp que el primero)
        None,               # S-05 Timestamp perdido
        '2024-02-10 09:30'
    ]
}
```

```

    'temperatura': [22.5, 21.0, None, 23.5, 20.0, 22.5, 19.5, 999.0], # 999.0
    ↪es un outlier brutal
    'humedad': [45, 50, 44, 48, None, 45, 52, -100] # -100 humedad imposible
}

df_iot = pd.DataFrame(data)
print("--- Dataset Raw (Sucio) ---")
display(df_iot)

```

--- Dataset Raw (Sucio) ---

	sensor_id	timestamp	temperatura	humedad
0	S-01	2024-02-10 09:00	22.5	45.0
1	S-02	2024-02-10 09:00	21.0	50.0
2	S-01	2024-02-10 09:15	NaN	44.0
3	S-03	2024-02-10 09:15	23.5	48.0
4	S-04	2024-02-10 09:15	20.0	NaN
5	S-01	2024-02-10 09:00	22.5	45.0
6	S-05	None	19.5	52.0
7	S-02	2024-02-10 09:30	999.0	-100.0

1.2.2 Ejercicio 1: Completitud

Problema: Hay sensores que han fallado y no han enviado temperatura o humedad (None/NaN). También hay un timestamp perdido.

Tarea: 1. Identifica cuántos valores nulos hay en cada columna. 2. Elimina las filas que no tengan timestamp (es crítico). 3. Para temperatura y humedad, decide si borrar o rellenar (puedes usar la media o un valor fijo).

```

[2]: # Identifico cuántos valores nulos hay en cada columna
print("Nulos por columna:")
print(df_iot.isnull().sum())

# Elimino filas sin timestamp
df_iot = df_iot.dropna(subset=["timestamp"])

# Calculo media ignorando valores imposibles ya que me daría una falsa media y
↪asi mantengo la tendencia natural de los datos
temp_valid = df_iot[(df_iot["temperatura"] >= -10) & (df_iot["temperatura"] <=
↪50)]
hum_valid = df_iot[(df_iot["humedad"] >= 0) & (df_iot["humedad"] <= 100)]
media_temp = temp_valid["temperatura"].mean()
media_hum = hum_valid["humedad"].mean()

# Relleno valores nulos con la media calculada y asi mantengo la completitud
↪sin distorsionar la serie temporal

```

```
df_iot["temperatura"] = df_iot["temperatura"].fillna(media_temp)
df_iot["humedad"] = df_iot["humedad"].fillna(media_hum)
display(df_iot)
```

Nulos por columna:

```
sensor_id      0
timestamp      1
temperatura     1
humedad         1
dtype: int64
```

	sensor_id	timestamp	temperatura	humedad
0	S-01	2024-02-10 09:00	22.5	45.0
1	S-02	2024-02-10 09:00	21.0	50.0
2	S-01	2024-02-10 09:15	21.9	44.0
3	S-03	2024-02-10 09:15	23.5	48.0
4	S-04	2024-02-10 09:15	20.0	46.4
5	S-01	2024-02-10 09:00	22.5	45.0
7	S-02	2024-02-10 09:30	999.0	-100.0

1.2.3 Ejercicio 2: Consistencia

Problema: La red a veces reenvía paquetes. El sensor S-01 parece tener una lectura duplicada a las 09:00.

Tarea: 1. Detecta los duplicados basándote en `sensor_id` y `timestamp`. 2. Elimínalos manteniendo solo la primera ocurrencia.

```
[3]: # Detecto los duplicados basándome en sensor_id + timestamp
duplicados = df_iot.duplicated(subset=["sensor_id", "timestamp"], keep="first")
print(f"Duplicados encontrados: {duplicados.sum()}")
display(df_iot[duplicados])

# Elimino duplicadas manteniendo solo la primera ocurrencia
df_iot = df_iot.drop_duplicates(subset=["sensor_id", "timestamp"], keep="first")
print("\nDataset sin duplicados:")
display(df_iot)
```

Duplicados encontrados: 1

	sensor_id	timestamp	temperatura	humedad
5	S-01	2024-02-10 09:00	22.5	45.0

Dataset sin duplicados:

	sensor_id	timestamp	temperatura	humedad
0	S-01	2024-02-10 09:00	22.5	45.0
1	S-02	2024-02-10 09:00	21.0	50.0

2	S-01	2024-02-10 09:15	21.9	44.0
3	S-03	2024-02-10 09:15	23.5	48.0
4	S-04	2024-02-10 09:15	20.0	46.4
7	S-02	2024-02-10 09:30	999.0	-100.0

1.2.4 Ejercicio 3: Precisión

Problema: * El sensor S-02 ha marcado **999.0°C**. ¡El edificio estaría en llamas! * La humedad de **-100%** es físicamente imposible.

Tarea: 1. Filtra el DataFrame para eliminar (o corregir) estos valores imposibles. * Temperatura válida: entre -10 y 50 °C * Humedad válida: entre 0 y 100 %

```
[4]: # Filtro el DataFrame para buscar un valor imposible
print("Filas con valores imposibles para borrar:")
fuera = df_iot[(df_iot["temperatura"] < -10) | (df_iot["temperatura"] > 50) |
               (df_iot["humedad"] < 0) | (df_iot["humedad"] > 100)]
display(fuera)

#Elimino filas con valores imposibles por que no puedo inventarme un valor.
df_iot = df_iot[(df_iot["temperatura"] >= -10) & (df_iot["temperatura"] <= 50) &
               (df_iot["humedad"] >= 0) & (df_iot["humedad"] <= 100)]
```

Filas con valores imposibles para borrar:

	sensor_id	timestamp	temperatura	humedad
7	S-02	2024-02-10 09:30	999.0	-100.0

1.2.5 Resultado Final

Muestra cómo ha quedado el DataFrame limpio y cuántas filas tiene ahora.

```
[5]: # Muestro como ha quedado el Dataframe limpio y en número de filas
print("DataFrame limpio:")
display(df_iot)
print("Número de filas: ",len(df_iot))
```

DataFrame limpio:

	sensor_id	timestamp	temperatura	humedad
0	S-01	2024-02-10 09:00	22.5	45.0
1	S-02	2024-02-10 09:00	21.0	50.0
2	S-01	2024-02-10 09:15	21.9	44.0
3	S-03	2024-02-10 09:15	23.5	48.0
4	S-04	2024-02-10 09:15	20.0	46.4

Número de filas: 5

02_Tarea_Riesgos_Integridad

March 10, 2026

1 Tarea Práctica: Riesgos de Integridad en Entornos Distribuidos

1.1 Contexto

Trabajas como Analista de Datos en un *E-commerce* de rápido crecimiento. El sistema de procesamiento de datos es distribuido: eventos de navegación y transacciones llegan continuamente desde microservicios separados (inventario, carrito, pasarela de pagos) a través de un sistema de mensajería (como Kafka).

Al ser una arquitectura distribuida (con retardos de red, reintentos automáticos y equipos de desarrollo separados), empiezan a aparecer comportamientos anómalos en los registros analíticos diarios.

1.2 Objetivo

Tu objetivo no es la simple “limpieza por valores atípicos” (como hicimos en IoT), sino reparar **riesgos críticos de integridad técnica y lógica** en un *subset* representativo de los datos.

Debes abordar los siguientes problemas extraídos de la **UT3 - Capítulo 2**: 1. **Duplicados y Replays**: El broker de mensajería reintentará envíos si falla la red, inyectando transacciones repetidas. 2. **Estados Parciales**: Fallos en el pipeline de enriquecimiento hacen que algunas compras lleguen sin asociar al usuario (`user_id`). 3. **Corrupción Lógica Silenciosa**: Un cambio silencioso de esquema en origen provoca que números lleguen como textos con formato europeo (ej: "14,50 €"), rompiendo los cálculos y devolviendo NaN tras conversiones fallidas.

1.2.1 ¡IMPORTANTE!

Como siempre, **justifica** por qué tomas cada decisión (¿borras un nulo? ¿rellenas? ¿te fías de qué columna para borrar duplicados?). Si delegas la justificación exclusivamente a una IA y no refleja tu entendimiento de la teoría, se notará y más cuando te pregunte en clase. No me seas, y se curioso y pierde un poco el tiempo en aprender.

```
[1]: import pandas as pd
import numpy as np

# --- CÓDIGO DE GENERACIÓN DE DATOS (NO MODIFICAR) ---
# Tienes una muestra de 8 eventos de la última hora en la Capa Bronce (Raw).

data = {
```

```

    'event_id': ['EV-100', 'EV-101', 'EV-101', 'EV-102', 'EV-103', 'EV-104', 'EV-105', 'EV-106'],
    'timestamp': [
        '2026-03-01 14:00:00',
        '2026-03-01 14:05:00',
        '2026-03-01 14:05:00', # Reintento exacto del EV-101
        '2026-03-01 14:10:00',
        '2026-03-01 14:12:00',
        '2026-03-01 14:15:00',
        '2026-03-01 14:10:00', # Evento rezagado (old timestamp)
        '2026-03-01 14:20:00'
    ],
    'user_id': ['U-01', 'U-02', 'U-02', None, 'U-03', 'U-01', 'U-04', 'U-05'],
    'action': ['view', 'purchase', 'purchase', 'purchase', 'view', 'purchase', 'view', 'purchase'],
    'amount': [0.0, 50.5, 50.5, 120.0, 0.0, "25,99 €", 0.0, -10.0] # Falla esquema y valor negativo
}

df_events = pd.DataFrame(data)
print("--- Dataset Raw (Sucio) extraído de Data Lake ---")
display(df_events)

```

--- Dataset Raw (Sucio) extraído de Data Lake ---

	event_id	timestamp	user_id	action	amount
0	EV-100	2026-03-01 14:00:00	U-01	view	0.0
1	EV-101	2026-03-01 14:05:00	U-02	purchase	50.5
2	EV-101	2026-03-01 14:05:00	U-02	purchase	50.5
3	EV-102	2026-03-01 14:10:00	None	purchase	120.0
4	EV-103	2026-03-01 14:12:00	U-03	view	0.0
5	EV-104	2026-03-01 14:15:00	U-01	purchase	25,99 €
6	EV-105	2026-03-01 14:10:00	U-04	view	0.0
7	EV-106	2026-03-01 14:20:00	U-05	purchase	-10.0

1.2.2 Ejercicio 1: Idempotencia y Duplicados por Reintentos (Replays)

(Ref: UT3 punto 2.1c)

Problema: En sistemas distribuidos, si el consumidor no confirma haber recibido la transacción EV-101, el productor la reenvía “por si acaso”. Si sumamos directamente los ingresos, estaríamos contando más dinero del real.

Tarea: 1. Detecta qué fila está duplicada, garantizando la idempotencia basándote exclusivamente en el identificador único de la transacción (`event_id`). 2. Elimina la fila fantasma manteniendo de forma segura la original.

```
[2]: # Detecto filas está duplicadas según su event_id
duplicadas = df_events.duplicated(subset=["event_id"], keep="first")
print(f"Replays detectados: {duplicadas.sum()}")
display(df_events[duplicadas])

# Elimino filas fantasmas manteniendo segura la forma original
df_events = df_events.drop_duplicates(subset=["event_id"], keep="first")
print("\nDataset sin replays:")
display(df_events)
```

Replays detectados: 1

	event_id	timestamp	user_id	action	amount
2	EV-101	2026-03-01 14:05:00	U-02	purchase	50.5

Dataset sin replays:

	event_id	timestamp	user_id	action	amount
0	EV-100	2026-03-01 14:00:00	U-01	view	0.0
1	EV-101	2026-03-01 14:05:00	U-02	purchase	50.5
3	EV-102	2026-03-01 14:10:00	None	purchase	120.0
4	EV-103	2026-03-01 14:12:00	U-03	view	0.0
5	EV-104	2026-03-01 14:15:00	U-01	purchase	25,99 €
6	EV-105	2026-03-01 14:10:00	U-04	view	0.0
7	EV-106	2026-03-01 14:20:00	U-05	purchase	-10.0

1.2.3 Ejercicio 2: Estados Parciales y Registros Huérfanos

(Ref: UT3 punto 2.1e)

Problema: El evento EV-102 es una compra real (amount = 120.0) pero se ha quedado en **estado parcial**. Existe, pero le falta el user_id asociado porque un microservicio que enriquece los datos de cliente estaba caído.

Tarea: 1. A diferencia del caso del “Edificio Inteligente” donde la falta de temperatura se podía interpolar, imputar un user_id a la ligera inventándolo puede arruinar métricas críticas como el Ticket Medio por Usuario o cruces de facturación. 2. Demuestra cómo identificarías las compras “huérfanas” (action == 'purchase' sin user_id). 3. Elimina o mueve ese registro a un dataframe de “cuarentena” justificando tu elección.

```
[3]: #Identifico compras huérfanas
compras_huorfanos = df_events[(df_events["action"] == "purchase") &
    ↪(df_events["user_id"].isnull())]
print("Compras huérfanas: ", len(compras_huorfanos))
display(compras_huorfanos)

# Muevo compras huérfanas a un DataFrame de cuarentena, así quedan registradas
↪y no se eliminan
```



```
df_cuarentena = compras_huerfanas.copy()
df_events = df_events[~((df_events["action"] == "purchase") &
    ↳(df_events["user_id"].isnull()))]

print("DataFrame sin huerfanas:")
display(df_events)
print("Dataframe con compras huerfanas en cuarentena:")
display(df_cuarentena)
```

Compras huerfanas: 1

	event_id	timestamp	user_id	action	amount
3	EV-102	2026-03-01 14:10:00	None	purchase	120.0

DataFrame sin huerfanas:

	event_id	timestamp	user_id	action	amount
0	EV-100	2026-03-01 14:00:00	U-01	view	0.0
1	EV-101	2026-03-01 14:05:00	U-02	purchase	50.5
4	EV-103	2026-03-01 14:12:00	U-03	view	0.0
5	EV-104	2026-03-01 14:15:00	U-01	purchase	25,99 €
6	EV-105	2026-03-01 14:10:00	U-04	view	0.0
7	EV-106	2026-03-01 14:20:00	U-05	purchase	-10.0

Dataframe con compras huerfanas en cuarentena:

	event_id	timestamp	user_id	action	amount
3	EV-102	2026-03-01 14:10:00	None	purchase	120.0

1.2.4 Ejercicio 3: Cambio de Esquema y Corrupción Silenciosa Lógica

(Ref: UT3 punto 2.2c y 2.1b)

Problema: Uno de los equipos front-end hizo una actualización silenciosa y empezó a enviar los importes concatenados con texto español ("25,99 €") en la compra EV-104. Al mezclarse números y Strings `amount` pasa a ser una columna `Object`. También hay un valor sin sentido (-10.0).

Tarea: 1. Muestra el tipo de dato (`dtypes`) actual de la columna `amount`. 2. Repara la columna: limpia el texto (cambia la coma por punto y quita el " €") solo en las celdas que sean texto, o fuerza a que toda la columna se convierta a `float`. `pd.to_numeric(errors='coerce')` puede ser tu gran aliado táctico. 3. Las compras no pueden tener costes negativos. Localiza el valor numérico contaminado y arréglalo pasándolo a 0 o borrando la fila.

```
[4]: # Muestro el tipo de dato (dtypes) actual de la columna amount.
print("Tipo actual de amount:", df_events["amount"].dtype)

# Reparo la columna, quito € y cambio coma por punto
def limpiar_amount(valor):
    if isinstance(valor, str):
        valor = valor.replace("€", "").replace(",", ".").strip()
```

```

    return valor

df_events["amount"] = df_events["amount"].apply(limpiar_amount)

# Fuerzo a que toda la columna se convierta a float. pd.
↳ to_numeric(errors='coerce')
df_events["amount"] = pd.to_numeric(df_events["amount"], errors="coerce")
print("Tipo reparado:", df_events["amount"].dtype)

# Localizo compras negativas y las paso a =
df_events.loc[df_events["amount"] < 0, "amount"] = 0.0

print("Dataset reparado:")
display(df_events)

```

Tipo actual de amount: object

Tipo reparado: float64

Dataset reparado:

	event_id	timestamp	user_id	action	amount
0	EV-100	2026-03-01 14:00:00	U-01	view	0.00
1	EV-101	2026-03-01 14:05:00	U-02	purchase	50.50
4	EV-103	2026-03-01 14:12:00	U-03	view	0.00
5	EV-104	2026-03-01 14:15:00	U-01	purchase	25.99
6	EV-105	2026-03-01 14:10:00	U-04	view	0.00
7	EV-106	2026-03-01 14:20:00	U-05	purchase	0.00

1.2.5 Resultado Final Consolidado

Prueba a sumar todos los amount de la columna de las compras resultantes. Tu contabilidad debería ser perfecta ahora.

```

[5]: # Filtro sólo lo que son compras
compras = df_events[df_events['action'] == 'purchase']
display(compras[['event_id', 'user_id', 'action', 'amount']])
print("Número de compras :", len(compras))
# Muestro la suma resultante
print("Total ingresos: ", compras['amount'].sum(), "€")

```

	event_id	user_id	action	amount
1	EV-101	U-02	purchase	50.50
5	EV-104	U-01	purchase	25.99
7	EV-106	U-05	purchase	0.00

Número de compras : 3

Total ingresos: 76.49 €

03_Tarea_Mecanismos_Verificacion

March 10, 2026

1 Tarea Práctica: Mecanismos de Verificación y Control de Integridad

1.1 Contexto

Estás al mando de la plataforma de datos de una gran red de logística. En el sistema distribuido intervienen camiones, almacenes, una pasarela de pagos y el Data Lake central. Como vimos en la **UT3 - Capítulo 3**, no puedes simplemente “confiar” en que los datos viajan bien. Necesitas **verificar**, establecer **controles de frontera** y aplicar mecánicas de **cuarentena** y **reconciliación** constantes.

1.2 Objetivo

Deberás auditar tres conjuntos de datos que acaban de llegar a la capa de ingesta, aplicando los conceptos técnicos de la teoría.

Debes abordar los siguientes puntos de control: 1. **Validación de Frontera y Cuarentena:** Filtrar transacciones imposibles (cantidades o importes negativos, nulos) ANTES de que entren a la zona Silver, aislando los errores. 2. **Verificación por Hash (Checksums):** Detectar corrupción de “man in the middle” o fallos de disco comprobando que el hash del archivo original corresponde con los datos recibidos. 3. **Auditoría de Reconciliación:** Cruzar los datos del almacén propio con una fuente externa de confianza (*Source of Truth*) para cuadrar el balance.

1.2.1 ¡IMPORTANTE!

Recuerda **justificar tus decisiones**. YO TU PROFESOR evaluará *por qué* escogiste esa estrategia basada en la teoría, no solo si el código de Pandas corre sin errores.

```
[1]: import pandas as pd
import hashlib

# --- CÓDIGO DE GENERACIÓN DE DATOS (NO MODIFICAR) ---

# Dataset 1: Ingesta de Almacén (Para validar perímetro)
data_ingesta = {
    'order_id': ['0-001', '0-002', '0-003', '0-004', '0-005'],
    'product': ['Laptop', 'Mouse', 'Teclado', None, 'Monitor'],
    'quantity': [1, -5, 2, 1, 0], # Hay errores obvios de validación aquí
    'price': [1200.0, 25.0, 45.0, 10.0, 300.0]
```

```

}
df_ingesta = pd.DataFrame(data_ingesta)

# Dataset 2: Archivo con Checksum MD5 incrustado (Para integridad de
↳transmisión)
# Simularemos que el proveedor nos envió un hash en metadatos para toda la
↳cadena de texto de la transacción.
data_transito = {
    'tx_id': ['TX-1', 'TX-2', 'TX-3'],
    'data_payload': ['userA:500:OK', 'userB:150:OK', 'userC:999:ERROR'],
    'expected_md5': [
        hashlib.md5('userA:500:OK'.encode()).hexdigest(),
        hashlib.md5('userB:150:OK'.encode()).hexdigest(), # Este hash cuadra
        hashlib.md5('userC:900:ERROR'.encode()).hexdigest() # El proveedor hizo
↳hash a 900, pero nos llegó 999. ¡CORRUPCIÓN!
    ]
}
df_transito = pd.DataFrame(data_transito)

# Dataset 3: Reconciliación
# Supongamos compras de la web vs cobros reales en el banco (Stripe)
data_web = pd.DataFrame({
    'cart_id': ['C-01', 'C-02', 'C-03'],
    'status_web': ['DELIVERED', 'DELIVERED', 'PENDING']
})

data_banco = pd.DataFrame({
    'cart_id': ['C-01', 'C-03', 'C-04'],
    'status_bank': ['PAID', 'PAID', 'PAID']
})

```

1.2.2 Ejercicio 1: Validación de Frontera (Schema Boundary) y Cuarentena (Dead-Letter)

(Ref: UT3 puntos 3.3b y 3.5a)

Problema: Muchos errores nacen en la entrada. Si metes datos donde la `quantity` es negativa o el `product` es Nulo a las tablas analíticas, romperás los dashboards.

Tarea: 1. Escribe condiciones de validación (Ej: `quantity > 0` y `product` distinto de nulo). 2. Separa el DataFrame `df_ingesta` original en **dos partes**: * `df_silver_limpio`: La “zona segura” que superó el chequeo. * `df_cuarentena_dead_letter`: Aquellos registros defectuosos que no pasaron la validación, para poder investigarlos en el futuro sin perderlos.

```

[2]: # Condiciones de validación: quantity > 0 y product distinto de nulo
mask_valido = (df_ingesta["quantity"] > 0) & (df_ingesta["product"].notna())

```

```
# Separo el DataFrame en dos partes: silver y cuarentena
df_silver_limpio = df_ingesta[mask_valido].copy()
df_cuarentena_dead_letter = df_ingesta[~mask_valido].copy()

print("En Silver: ", len(df_silver_limpio))
display(df_silver_limpio)
print("En Cuarentena: ", len(df_cuarentena_dead_letter))
display(df_cuarentena_dead_letter)
```

En Silver: 2

	order_id	product	quantity	price
0	0-001	Laptop	1	1200.0
2	0-003	Teclado	2	45.0

En Cuarentena: 3

	order_id	product	quantity	price
1	0-002	Mouse	-5	25.0
3	0-004	None	1	10.0
4	0-005	Monitor	0	300.0

1.2.3 Ejercicio 2: Uso de Hashes/Checksums para Detectar Corrupción de Contenido

(Ref: UT3 punto 3.1)

Problema: Una transferencia de red se interrumpió o el disco tuvo un pico magnético (Data Corruption). A simple vista, “todo está ahí”.

Tarea: 1. Crea una nueva columna en `df_transito` llamada `calculated_md5` donde calcules el hash MD5 de la columna `data_payload`. (Pista: Usa un *apply* de pandas con `hashlib.md5(str(x).encode()).hexdigest()`) 2. Compara `calculated_md5` con `expected_md5`. 3. Filtra y muestra por pantalla exclusivamente la transacción que esté **Corrupta** (donde el checksum no cuadre).

```
[3]: # Calculo el hash MD5 de data_payload
df_transito["calculated_md5"] = df_transito["data_payload"].apply(
    lambda x: hashlib.md5(str(x).encode()).hexdigest()
)

# Comparo calculated_md5 con expected_md5
df_transito["integridad_ok"] = df_transito["calculated_md5"] == df_transito["expected_md5"]
print("MD5 esperado versus calculado")
display(df_transito[["tx_id", "data_payload", "expected_md5", "calculated_md5", "integridad_ok"]])
```

```
# Filtro y muestro las transacciones corruptas
corruptas = df_transito[~df_transito["integridad_ok"]]
print("Transacciones corruptas: ", len(corruptas))
display(corruptas[["tx_id", "data_payload"]])
```

MD5 esperado versus calculado

	tx_id	data_payload	expected_md5	\
0	TX-1	userA:500:OK	3e573889c836566157641e89023346ac	
1	TX-2	userB:150:OK	e9ec9b608384fee5171f781d7ff70b9a	
2	TX-3	userC:999:ERROR	d3ccfca140894ab834832aeb65cf3c85	

		calculated_md5	integridad_ok
0	3e573889c836566157641e89023346ac		True
1	e9ec9b608384fee5171f781d7ff70b9a		True
2	2e3f4e90bfaf169ccf834bb5569eb2a5		False

Transacciones corruptas: 1

	tx_id	data_payload
2	TX-3	userC:999:ERROR

1.2.4 Ejercicio 3: Auditorías de Consistencia y Reconciliación (*Source of Truth*)

(Ref: UT3 puntos 3.4b y 3.4c)

Problema: La base de datos dice una cosa y el banco otra (clásico en distribución eventual).

Tarea: Haz un **outer join** (o merge en Pandas) usando **cart_id** entre **data_web** y **data_banco**. Luego responde mediante código y comentarios: 1. ¿A qué carrito se le ha enviado mercancía (**status_web** = **DELIVERED**) pero resulta que NO consta como pagado en el banco (no existe en **data_banco**)? ¡Alarma de fraude/pérdida! 2. ¿Qué carrito aparece como pagado en el banco (**PAID**) pero nuestra tienda online ni siquiera sabe que existe (no está en **data_web**)? ¡Alarma de datos huérfanos!

```
[4]: # Hago el outer join
df_reconciliacion = pd.merge(data_web, data_banco, on="cart_id", how="outer")
print("Outer join:")
display(df_reconciliacion)

# Enviado pero no pagado -> ¡Alarma de fraude/pérdida!
enviado_no_pagado = df_reconciliacion[
    (df_reconciliacion["status_web"] == "DELIVERED") &
    (df_reconciliacion["status_bank"].isna())
]
print("ALARMA - Enviado sin pago en banco:")
display(enviado_no_pagado)

# Pagado pero no en tienda online -> ¡Alarma de datos huérfanos!
```

```

pagado_no_en_web = df_reconciliacion[
    (df_reconciliacion["status_bank"] == "PAID") &
    (df_reconciliacion["status_web"].isna())
]
print("ALARMA - Pagado pero en tienda online no existe:")
display(pagado_no_en_web)

```

Outer join:

	cart_id	status_web	status_bank
0	C-01	DELIVERED	PAID
1	C-02	DELIVERED	NaN
2	C-03	PENDING	PAID
3	C-04	NaN	PAID

ALARMA - Enviado sin pago en banco:

	cart_id	status_web	status_bank
1	C-02	DELIVERED	NaN

ALARMA - Pagado pero en tienda online no existe:

	cart_id	status_web	status_bank
3	C-04	NaN	PAID

04_Tarea_Gestion_Integridad

March 10, 2026

1 Tarea Práctica: Gestión y Roles en la Integridad de Datos

1.1 Contexto

Has ascendido a Data Engineer Senior en una plataforma de Streaming de Video. Tu empresa maneja millones de eventos (play, pause, stop) emitidos por distintas aplicaciones (Smart TVs, Móviles, Web).

Tal y como estudiaste en la **UT3 - Capítulo 4**, la integridad no se logra solo con tecnología, sino con **gestión, contratos y responsabilidades claras**. Tu equipo (la capa de *Procesamiento y Catálogo*) no puede limpiar infinitamente la basura que envían los productores, y el equipo de BI (*Consumo*) no puede usar datos que no están certificados.

1.2 Objetivo

Debes aplicar **Protocolos de Respuesta y Coherencia** sobre un lote de datos recién ingerido, decidiendo qué hacer con los problemas basándote en roles y responsabilidades.

Abordarás 3 retos típicos de gestión: 1. **Responsabilidad del Productor (Garbage In)**: Lidiar con eventos que violan el contrato mínimo indispensable para ser procesados. 2. **Protocolo de SLA y Late Events**: Gestionar eventos que rompieron el acuerdo de nivel de servicio (llegaron demasiado tarde para el cierre contable diario). 3. **Zonas de Datos (Raw a Refined)**: Promocionar datos de la capa Bronce a Plata haciendo cumplir un contrato de tipos estricto y separando la corrupción a Cuarentena.

1.2.1 ¡IMPORTANTE!

En esta práctica se evaluará tu capacidad de razonar, aunque la IA te de ese código. Debes **justificar** cada acción en un comentario de texto argumentando en base a la teoría de Roles y Protocolos de la UT3. En resumen, intenta ser curioso para aprender.

```
[1]: import pandas as pd

# --- CÓDIGO DE GENERACIÓN DE DATOS (NO MODIFICAR) ---
# Simulamos un Data Lake (Zona Raw/Bronce) donde acaba de volcarse la "bolsa"
# de eventos de ayer y hoy por la mañana.

data_lake_raw = {
    'event_id': ['EV-01', 'EV-02', None, 'EV-04', 'EV-05', 'EV-06', 'EV-07'], #
    # EV-03 perdió su ID en la App Móvil
```



```

    'app_source': ['iOS', 'Android', 'Web', 'iOS', 'SmartTV', 'iOS', 'Android'],
    'event_type': ['play', 'pause', 'play', 'stop', 'play', 'play', 'play'],
    'timestamp_event': [
        '2026-03-01 22:30:00', # Ayer noche (OK)
        '2026-03-01 23:15:00', # Ayer noche (OK)
        '2026-03-01 23:45:00', # Ayer noche, pero sin event_id (Violación de
↳contrato)
        '2026-03-02 01:00:00', # Madrugada de hoy (Evento normal de hoy)
        '2026-03-01 20:00:00', # Ayer noche, pero LLEGÓ TARDE a la capa de
↳ingesta (Late Event)
        '2026-03-02 08:30:00', # Hoy por la mañana
        '2026-03-02 09:00:00'
    ],
    'ingestion_time': [ # Cuándo lo recibió el servidor
        '2026-03-01 22:30:05',
        '2026-03-01 23:15:10',
        '2026-03-01 23:45:05',
        '2026-03-02 01:00:05',
        '2026-03-02 08:45:00', # ¡FÍJATE! El evento del SmartTV de ayer se
↳quedó sin Wi-Fi y llegó al servidor hoy a las 8:45 AM.
        '2026-03-02 08:30:05',
        '2026-03-02 09:00:02'
    ],
    'seconds_watched': [1200.0, 45.0, 300.0, 10.0, 5000.0, "ochenta", 60.0] #
↳Un desarrollador de iOS mandó texto. ¡Corrupción silenciosa!
}

df_raw = pd.DataFrame(data_lake_raw)
# Convertimos strings a datetime para facilitar el filtrado
df_raw['timestamp_event'] = pd.to_datetime(df_raw['timestamp_event'])
df_raw['ingestion_time'] = pd.to_datetime(df_raw['ingestion_time'])

print("--- Data Lake (Capa Raw / Bronce) ---")
display(df_raw)

```

--- Data Lake (Capa Raw / Bronce) ---

	event_id	app_source	event_type	timestamp_event	ingestion_time \
0	EV-01	iOS	play	2026-03-01 22:30:00	2026-03-01 22:30:05
1	EV-02	Android	pause	2026-03-01 23:15:00	2026-03-01 23:15:10
2	None	Web	play	2026-03-01 23:45:00	2026-03-01 23:45:05
3	EV-04	iOS	stop	2026-03-02 01:00:00	2026-03-02 01:00:05
4	EV-05	SmartTV	play	2026-03-01 20:00:00	2026-03-02 08:45:00
5	EV-06	iOS	play	2026-03-02 08:30:00	2026-03-02 08:30:05
6	EV-07	Android	play	2026-03-02 09:00:00	2026-03-02 09:00:02

	seconds_watched
0	1200.0

1	45.0
2	300.0
3	10.0
4	5000.0
5	ochenta
6	60.0

1.2.2 Ejercicio 1: Responsabilidad del Productor (El contrato mínimo)

(Ref: UT3 puntos 4.1a y 4.1f)

Problema: La teoría dice que la responsabilidad del Productor (las Apps) es generar IDs únicos y trazables. Si un evento no tiene `event_id`, no podemos saber si es un duplicado en el futuro. BI (Capa de consumo) no puede usar basura.

Tarea: 1. Filtra y localiza el/los eventos que violan esta regla fundamental (aquellos con `event_id` nulo). 2. Sepáralos en un DataFrame `df_rechazados_origen` (para quejarnos con el equipo de front-end). 3. Mantén el resto en `df_raw_validado`.

```
[2]: # Filtro los eventos con event_id nulo
sin_id = df_raw["event_id"].isnull()
print("Eventos sin event_id: ", sin_id.sum())
display(df_raw[sin_id])

# Separo los eventos con event_id nulo en un Dataframe de rechazados
df_rechazados_origen = df_raw[sin_id].copy()

# El resto de eventos los guardo en un raw validado
df_raw_validado = df_raw[~sin_id].copy()

print("Validados: ", len(df_raw_validado))
display(df_raw_validado)
```

Eventos sin event_id: 1

	event_id	app_source	event_type	timestamp_event	ingestion_time	\
2	None	Web	play	2026-03-01 23:45:00	2026-03-01 23:45:05	

	seconds_watched
2	300.0

Validados: 6

	event_id	app_source	event_type	timestamp_event	ingestion_time	\
0	EV-01	iOS	play	2026-03-01 22:30:00	2026-03-01 22:30:05	
1	EV-02	Android	pause	2026-03-01 23:15:00	2026-03-01 23:15:10	
3	EV-04	iOS	stop	2026-03-02 01:00:00	2026-03-02 01:00:05	
4	EV-05	SmartTV	play	2026-03-01 20:00:00	2026-03-02 08:45:00	
5	EV-06	iOS	play	2026-03-02 08:30:00	2026-03-02 08:30:05	

6 EV-07 Android play 2026-03-02 09:00:00 2026-03-02 09:00:02

```
seconds_watched
0      1200.0
1       45.0
3       10.0
4      5000.0
5      ochenta
6       60.0
```

1.2.3 Ejercicio 2: Protocolo SLA y Late Events (Consistencia eventual)

(Ref: UT3 punto 4.3 Ejemplo 2 y 4.2b)

Problema: En la empresa hay un contrato de SLA: “El cierre diario de métricas de ayer se consolida a las 08:00 AM de hoy. Cualquier evento de ayer que ingrese al almacén DESPUÉS de las 08:00 AM será procesado por separado para no cambiar los dashboards ya publicados”.

Tarea: Revisa el `df_raw_validado` (ya sin nulos). 1. Encuentra el “Late Event”: Un evento cuyo `timestamp_event` indique que ocurrió “ayer” (día 01), pero que su `ingestion_time` indique que nuestro servidor lo recibió “hoy” (día 02) DESPUÉS del umbral de las 08:00 AM. 2. Separa ese evento en un `df_late_events` para un reproceso contable secundario, y deja el resto listos en `df_para_silver`.

```
[3]: # Busco el último evento con timestamp_event con fecha de ayer, día 1, y con un
      ↪ ingestion_time de hoy, día 2, después de las 8 AM
umbral_sla = pd.Timestamp("2026-03-02 08:00:00")
dia_ayer = pd.Timestamp("2026-03-01").date()

mask_late = (
    (df_raw_validado["timestamp_event"].dt.date == dia_ayer) &
    (df_raw_validado["ingestion_time"] > umbral_sla)
)

# Separo el evento para un reproceso contable secundario y el resto los dejo en
↪ silver
df_late_events = df_raw_validado[mask_late].copy()
df_para_silver = df_raw_validado[~mask_late].copy()

print("Para un proceso contable secundario: ", len(df_late_events))
display(df_late_events)
print("Listos en Silver: ", len(df_para_silver))
display(df_para_silver)
```

Para un proceso contable secundario: 1

```
event_id app_source event_type      timestamp_event      ingestion_time \
4      EV-05      SmartTV      play 2026-03-01 20:00:00 2026-03-02 08:45:00
```

```
seconds_watched
4          5000.0
```

Listos en Silver: 5

	event_id	app_source	event_type	timestamp_event	ingestion_time	\
0	EV-01	iOS	play	2026-03-01 22:30:00	2026-03-01 22:30:05	
1	EV-02	Android	pause	2026-03-01 23:15:00	2026-03-01 23:15:10	
3	EV-04	iOS	stop	2026-03-02 01:00:00	2026-03-02 01:00:05	
5	EV-06	iOS	play	2026-03-02 08:30:00	2026-03-02 08:30:05	
6	EV-07	Android	play	2026-03-02 09:00:00	2026-03-02 09:00:02	

```
seconds_watched
0          1200.0
1           45.0
3           10.0
5         ochenta
6           60.0
```

1.2.4 Ejercicio 3: Certificación y Prevención (Hacia la Capa Silver)

(Ref: UT3 puntos 4.2c, 4.3 Ejemplo 3 y 4.2d)

Problema: Toca pasar `df_para_silver` de la zona Raw a la zona Refined (Plata). El contrato dice que la columna `seconds_watched` debe ser estrictamente numérica (Float) para poder sumar los minutos de visualización. Sin embargo, un update de la app de iOS ha colado texto ("ochenta").

Tarea: 1. Intenta forzar la conversión de toda la columna `seconds_watched` a numérico usando `pd.to_numeric()`. Usa el parámetro que convierte los fallos imprevistos en NaN (para no romper en seco todo el pipeline). 2. Una vez hecho, el sistema de Detección debe aislar (Cuarentena) ese registro dañado para que no pase la certificación hacia Gold. Filtra los nulos en `seconds_watched` y guárdalos en `df_cuarentena`. 3. El dataframe resultante se llamará `df_silver_certificado` y estará completamente pulido. Muéstralo por pantalla.

```
[4]: # Fuerzo la conversión de seconds_watched a numérico
df_para_silver["seconds_watched"] = pd.to_numeric(
    df_para_silver["seconds_watched"], errors="coerce"
)

# Filtro los nulos y los guardo en cuarentena
mask_nulos = df_para_silver["seconds_watched"].isna()
df_cuarentena = df_para_silver[mask_nulos].copy()
print("Cuarentena: ", len(df_cuarentena))
display(df_cuarentena)

#Creo un DataFrame resultante limpio
df_silver_certificado = df_para_silver[~mask_nulos].copy()
```

